

第十六讲：Fourier 变换、离散 Fourier 变换与快速 Fourier 变换

本讲目标

本讲从连续 Fourier 变换出发，说明计算机中为什么自然得到离散 Fourier 变换 (Discrete Fourier Transform, DFT)，随后引入单位根和 Fourier 矩阵，解释 DFT 的矩阵形式、逆变换以及“多项式在单位根处求值”的视角。最后利用 Fourier 矩阵的递归结构推导快速 Fourier 变换 (Fast Fourier Transform, FFT)，说明其复杂度为什么可以从 $O(n^2)$ 降为 $O(n \log n)$ 。

连续 Fourier 变换

设 $f(x)$ 是定义在实数轴上的函数，其 Fourier 变换定义为

$$F(t) = \int_{-\infty}^{+\infty} f(x) e^{-2\pi i x t} dx. \quad (1)$$

这里 x 可以理解为时间域或空间域变量， t 是频率变量。Fourier 变换的核心作用是把一个函数从原始域表示转化为频率域表示，从而揭示函数中不同频率成分的强弱关系。

直观地看，式(1)用复指数基函数 $e^{-2\pi i x t}$ 去“探测” $f(x)$ 中频率为 t 的成分。当 $f(x)$ 与该频率的振荡模式匹配时，积分累积较大；当二者振荡不匹配时，正负抵消较强，积分值较小。

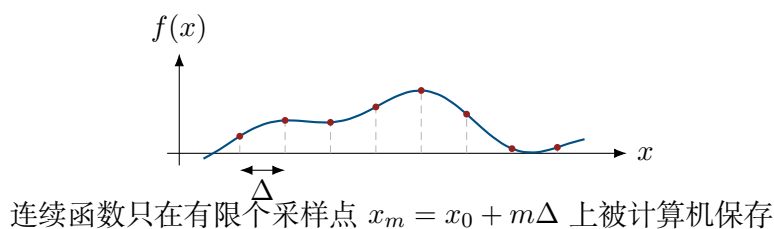


图 1 从连续函数到离散采样。计算机只能处理有限个采样点，积分需要被求和近似。

从连续 Fourier 变换到 DFT

在计算机中，既不能保存 $(-\infty, +\infty)$ 上的全部函数值，也不能精确计算无穷区间上的积分。因此需要把连续变量和积分同时离散化。取 N 个等间距采样点

$$x_m = x_0 + m\Delta, \quad m = 0, 1, \dots, N-1, \quad (2)$$

其中 Δ 是采样间隔。用 Riemann 求和近似式(1)，有

$$F(t) \approx \Delta \sum_{m=0}^{N-1} f(x_m) e^{-2\pi i x_m t}. \quad (3)$$

为了得到有限个频率点上的值，令

$$t_n = \frac{n}{N\Delta}, \quad n = 0, 1, \dots, N-1. \quad (4)$$

代入式(3) 可得

$$\begin{aligned} F(t_n) &\approx \Delta \sum_{m=0}^{N-1} f(x_0 + m\Delta) \exp\left(-2\pi i (x_0 + m\Delta) \frac{n}{N\Delta}\right) \\ &= \Delta \exp\left(-2\pi i \frac{x_0 n}{N\Delta}\right) \sum_{m=0}^{N-1} f(x_m) \exp\left(-2\pi i \frac{mn}{N}\right). \end{aligned} \quad (5)$$

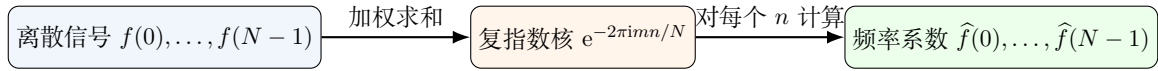
前面的因子

$$\Delta \exp\left(-2\pi i \frac{x_0 n}{N\Delta}\right) \quad (6)$$

只与采样起点、采样间隔和频率编号有关。若只关注离散变换的代数结构，核心部分就是

$$\hat{f}(n) = \sum_{m=0}^{N-1} f(m) \exp\left(-2\pi i \frac{mn}{N}\right), \quad n = 0, 1, \dots, N-1. \quad (7)$$

这就是 DFT 的标准形式之一。



DFT 可理解为：用一组离散复指数基，对离散信号做投影。

图 2 DFT 的计算结构。每个频率编号 n 对应一组复指数权重。

直接根据式(7) 计算全部 N 个频率值，需要对每个 n 做一次长度为 N 的求和，总复杂度为 $O(N^2)$ 。FFT 的目标就是利用这些复指数权重之间的重复结构，减少重复计算。

单位根与 Fourier 矩阵

令

$$\omega_n = e^{-2\pi i/n}. \quad (8)$$

则 $\omega_n^n = 1$ ，并且

$$1, \omega_n, \omega_n^2, \dots, \omega_n^{n-1} \quad (9)$$

是复平面上的 n 个 n 次单位根。它们均匀分布在单位圆上。

用 ω_n 可以把 DFT 写成更紧凑的形式：

$$y_j = \sum_{k=0}^{n-1} \omega_n^{jk} x_k, \quad j = 0, 1, \dots, n-1. \quad (10)$$

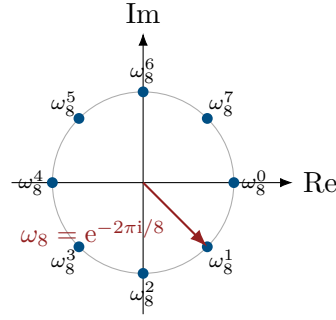


图 3 8 次单位根。采用负指数约定时，幂次沿顺时针方向前进。

若记

$$\mathbf{x} = (x_0, x_1, \dots, x_{n-1})^T, \quad \mathbf{y} = (y_0, y_1, \dots, y_{n-1})^T, \quad (11)$$

则 DFT 可写为矩阵乘法

$$\mathbf{y} = F_n \mathbf{x}, \quad (12)$$

其中

$$F_n = \left(\omega_n^{jk} \right)_{j,k=0}^{n-1} = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n & \omega_n^2 & \dots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \dots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \dots & \omega_n^{(n-1)^2} \end{pmatrix}. \quad (13)$$

这个矩阵称为 n 阶 Fourier 矩阵。

$$F_n = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)^2} \end{pmatrix}$$

第 j 行
对应频率 j

第 k 列
对应输入位置 k

图 4 Fourier 矩阵中，第 j 行给出频率 j 对所有输入点的权重。

Fourier 矩阵的逆

Fourier 矩阵不是 Hermite 矩阵，但它具有非常重要的正交性。记 F_n^* 为共轭转置，则

$$F_n^* F_n = n I_n. \quad (14)$$

因此

$$F_n^{-1} = \frac{1}{n} F_n^*. \quad (15)$$

这给出逆 DFT:

$$x_k = \frac{1}{n} \sum_{j=0}^{n-1} \omega_n^{-jk} y_j, \quad k = 0, 1, \dots, n-1. \quad (16)$$

式(14) 的本质是几何级数求和。对任意 $a, b \in \{0, 1, \dots, n-1\}$, 有

$$\sum_{k=0}^{n-1} \omega_n^{(a-b)k} = \begin{cases} n, & a = b, \\ 0, & a \neq b. \end{cases} \quad (17)$$

当 $a \neq b$ 时, ω_n^{a-b} 是非 1 的 n 次单位根, 有限几何级数正好相消。

DFT 的多项式求值

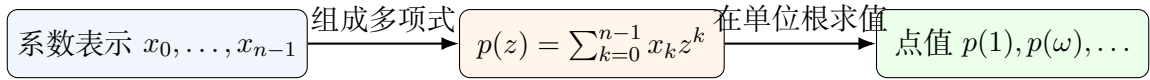
令输入向量 $\mathbf{x}$ 是一个次数小于 n 的多项式的系数:

$$p(z) = x_0 + x_1 z + x_2 z^2 + \dots + x_{n-1} z^{n-1}. \quad (18)$$

由式(10) 可知

$$y_j = \sum_{k=0}^{n-1} x_k (\omega_n^j)^k = p(\omega_n^j), \quad j = 0, 1, \dots, n-1. \quad (19)$$

因此, DFT 可以看成把多项式的“系数表示”转化为“在单位根处的点值表示”。逆 DFT 则把这些点值重新恢复为系数。



Fourier 矩阵乘法 $F_n \mathbf{x}$ 正是在这些特殊点上的批量求值。

图 5 DFT 的多项式解释: 系数表示 $\rightarrow$ 单位根处的点值表示。

这个视角对于理解 FFT 很关键。因为单位根之间存在递归关系: $2m$ 次单位根平方后变成 m 次单位根。也就是说, 高阶 Fourier 矩阵中包含低阶 Fourier 矩阵的结构。

FFT 的分治推导

设 $n = 2m$ 。把多项式 $p(z)$ 按偶数次项和奇数次项拆开:

$$\begin{aligned} p(z) &= \sum_{k=0}^{2m-1} x_k z^k \\ &= \sum_{k=0}^{m-1} x_{2k} z^{2k} + \sum_{k=0}^{m-1} x_{2k+1} z^{2k+1} \\ &= p_e(z^2) + z p_o(z^2), \end{aligned} \quad (20)$$

其中

$$p_e(u) = \sum_{k=0}^{m-1} x_{2k} u^k, \quad p_o(u) = \sum_{k=0}^{m-1} x_{2k+1} u^k. \quad (21)$$

令 $\omega_{2m} = e^{-2\pi i/(2m)}$ 。因为

$$\omega_{2m}^2 = \omega_m, \quad \omega_{2m}^m = -1, \quad (22)$$

所以对于 $j = 0, 1, \dots, m-1$, 有

$$\begin{aligned} y_j &= p(\omega_{2m}^j) \\ &= p_e(\omega_{2m}^{2j}) + \omega_{2m}^j p_o(\omega_{2m}^{2j}) \\ &= p_e(\omega_m^j) + \omega_{2m}^j p_o(\omega_m^j), \end{aligned} \quad (23)$$

并且

$$\begin{aligned} y_{j+m} &= p(\omega_{2m}^{j+m}) \\ &= p_e(\omega_m^j) - \omega_{2m}^j p_o(\omega_m^j). \end{aligned} \quad (24)$$

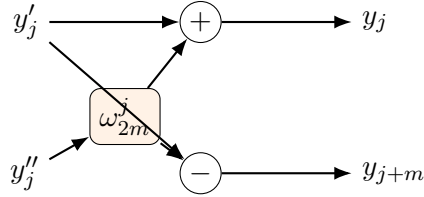
记

$$y'_j = p_e(\omega_m^j), \quad y''_j = p_o(\omega_m^j). \quad (25)$$

则一次 FFT 合并公式为

$$\begin{cases} y_j = y'_j + \omega_{2m}^j y''_j, \\ y_{j+m} = y'_j - \omega_{2m}^j y''_j, \end{cases} \quad j = 0, 1, \dots, m-1. \quad (26)$$

这就是 FFT 的基本蝶形运算。



先分别计算偶数项和奇数项的 m 点 DFT，再用一个旋转因子 ω_{2m}^j 合并成 $2m$ 点 DFT。

图 6 FFT 的蝶形合并公式。

矩阵分解形式

设 P_{2m} 是把输入向量重排为“偶数下标在前、奇数下标在后”的置换矩阵，即

$$P_{2m}(x_0, x_1, x_2, x_3, \dots, x_{2m-2}, x_{2m-1})^T = (x_0, x_2, \dots, x_{2m-2}, x_1, x_3, \dots, x_{2m-1})^T. \quad (27)$$

再令

$$D_m = \text{diag}(1, \omega_{2m}, \omega_{2m}^2, \dots, \omega_{2m}^{m-1}). \quad (28)$$

则 Fourier 矩阵有如下递归分解：

$$F_{2m} = \begin{pmatrix} I_m & D_m \\ I_m & -D_m \end{pmatrix} \begin{pmatrix} F_m & 0 \\ 0 & F_m \end{pmatrix} P_{2m}. \quad (29)$$

该式清楚说明了 FFT 的三步结构：先置换输入，把偶数项和奇数项分开；再递归计算两个 m 点 DFT；最后通过 D_m 中的旋转因子完成蝶形合并。

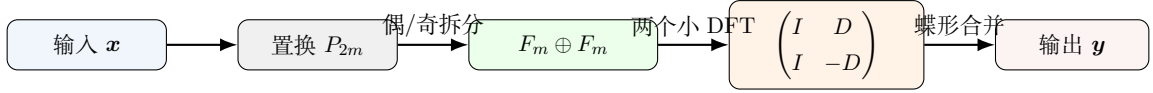


图 7 Fourier 矩阵的递归分解对应 FFT 的一次迭代。

以 F_4 为例

取 $n = 4$, $\omega_4 = e^{-2\pi i/4} = -i$, 则

$$F_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega_4 & \omega_4^2 & \omega_4^3 \\ 1 & \omega_4^2 & \omega_4^4 & \omega_4^6 \\ 1 & \omega_4^3 & \omega_4^6 & \omega_4^9 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{pmatrix}. \quad (30)$$

令 $m = 2$, 把输入拆成偶数部分 (x_0, x_2) 和奇数部分 (x_1, x_3) 。先分别计算两个 2 点 DFT:

$$F_2 \begin{pmatrix} x_0 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_0 + x_2 \\ x_0 - x_2 \end{pmatrix}, \quad F_2 \begin{pmatrix} x_1 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_1 + x_3 \\ x_1 - x_3 \end{pmatrix}. \quad (31)$$

再乘上旋转因子并合并, 即得到 4 点 DFT。这里已经可以看到 F_4 中包含两个 F_2 的结构。

复杂度分析

设 $T(n)$ 表示计算 n 点 DFT 所需的基本运算量。直接矩阵乘法需要

$$T_{\text{direct}}(n) = O(n^2). \quad (32)$$

FFT 每次把一个 n 点 DFT 分成两个 $n/2$ 点 DFT, 并在合并阶段做 $O(n)$ 次旋转因子乘法和加减法, 因此

$$T(n) = 2T(n/2) + O(n). \quad (33)$$

当 $n = 2^\ell$ 时, 递归深度为 $\ell = \log_2 n$, 每层总合并代价为 $O(n)$, 所以

$$T(n) = O(n \log n). \quad (34)$$

这正是 FFT 相比朴素 DFT 的关键优势。

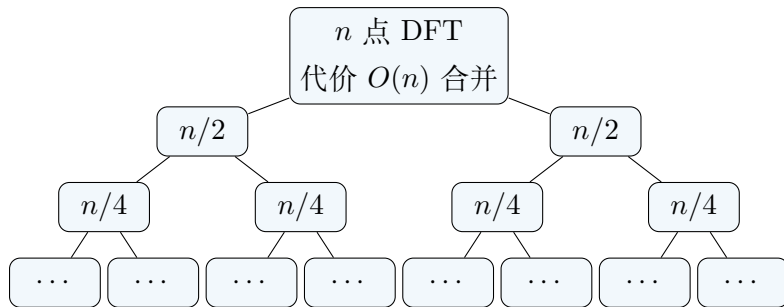


图 8 FFT 递归树。每一层的总合并代价为 $O(n)$, 共有 $\log_2 n$ 层。

小结

连续 Fourier 变换经过有限采样和有限频率网格后得到 DFT；DFT 可以写成 Fourier 矩阵乘法；Fourier 矩阵的行列由单位根生成，并满足 $F_n^{-1} = \frac{1}{n} F_n^*$ ；从多项式角度看，DFT 是在单位根处的批量求值；利用 $2m$ 次单位根与 m 次单位根之间的递归关系，可以把 $2m$ 点 DFT 拆成两个 m 点 DFT，并通过蝶形公式合并，由此得到 $O(n \log n)$ 的 FFT。